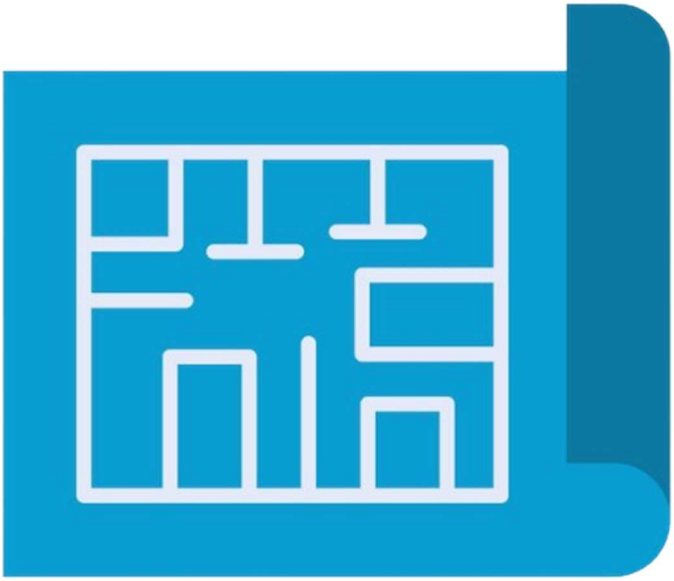
Blueprint

-

Projekt dokumentáció



Csapattagok

Gombár Martin – csapatvezető

13.a

Gémes Gergő

13.b

Tartalomjegyzék

Bevezetés ... 5
Téma ismertetése ... 5
A szoftver célja ... 5
Fejlesztői útmutató ... 6
Frontend technológiák ... 6
Backend technológiák ... 6
Infrastruktúra ... 6
Projekt struktúra ... 7
Backend ... 7
Fontos szerepek ... 7
Kiemelt kódrészlet: CORS origin normalizálás ... 8
Hitelesítés és jogosultságkezelés ... 8
Folyamat ... 8
Kiemelt kódrészlet: token és adatbázis-állapot összevonása ... 9
Adatbázis ... 10
Fő entitások ... 10
users tábla / user modell ... 10
events tábla / event modell és classroom ... 11
API végpontok és route-struktúra ... 12
7.1 Public/User route-ok ... 12
7.2 Event route-ok ... 12
7.3 Teacher route-ok ... 12
7.4 Admin route-ok ... 13
Eseménykezelés ... 13
8.1 Event lista + userhez kötött állapot ... 13
8.2 Jelentkezési szabályok ... 13

8.3 Kommentek AI moderációval 13
Admin felület..... 14
9.1 Fontos minta: teacher classroom feloldás appointmenthez 15
9.2 Event create/update követelmények 15
Tanári felület..... 15
10.1 Teacher profil 16
10.2 Saját időpontok 16
10.3 Availability szabályosság 16
10.4 Teacher event létrehozás..... 16
Frontend 17
11.1 App Router szemlélet..... 17
11.2 State és adatkezeles..... 17
EventDetailsView komponens..... 17
12.1 Tab-struktúra..... 17
12.2 React Query minta 18
Navigációs modul és útvonaltervezés 19
13.1 Koncepció 19
13.2 Mátrix reprezentacio..... 19
13.3 Név normalizálás és alias mapping..... 20
13.4 Lépcső kapcsolatok..... 20
13.5 Blokkolási logika..... 20
13.6 A* heuristika..... 21
13.7 UI/UX jellemzők..... 21
Tanterem-kezelés és jelölési rendszer..... 21
Biztonság és megbízhatóság..... 23
15.1 Rate limiting 23
15.2 Email verification gate 23

15.3 Soft delete és status gate ... 23
15.4 AI moderation fallback ... 23
Hibakezelés ... 23
16.1 Backend ... 23
16.2 Frontend ... 23
Teljesítményi szempontok ... 23
17.1 Query invalidálás ... 23
17.2 Navigáció algoritmus ... 23
17.3 Prisma ... 24
A rendszer telepítése és futtatása ... 25
Frontend és backend indítása ... 25
Adatbázis migráció és Prisma frissítés ... 25
Tervezési döntések és indoklás ... 25
19.1 Miért mátrix alapú navigáció? ... 25
19.2 Miért kötelező classroom eventnél? ... 25
19.3 Miért role + status + deletedAt gate? ... 26
Kódrészlet-gyűjtemény ... 27
21.1 Route mountolás (backend) ... 27
21.2 Auth user gate ... 27
21.3 Event classroom kötelezettség ... 27
21.4 A* útvonalkeresés mag ... 27
Fogalomtár ... 28
Tesztelés ... 29
Tesztelési stratégia ... 29
Backend automatizált tesztek ... 29
Tesztek helye: backend/tests/*.test.js ... 29
Futtatási parancs: cd backend && npm test ... 29

Lefedett tipikus backend esetek 29
Frontend e2e smoke tesztek (Playwright) 30
Tesztek helye: frontend/e2e/*.spec.ts és frontend/e2e/setup/auth.setup.ts 30
Futtatási parancsok: 30
Főbb ellenőrzött e2e esetek 30
Tesztelői hozzáférés és előfeltételek 30
Hibakezelési fókusz 31
Fontosabb fájlok röviden 32
24.1 Backend - kulcsfájlok 32
24.2 Frontend - kulcsfájlok 32
24.3 Route fájlok (fontosabb) 33
24.4 Dokumentáció 33
Mini működési példák 34
25.1 Auth gate példa (backend) 34
25.2 Event classroom követelmény példa 34
25.3 A* útvonaltervezés példa (frontend) 34
25.4 TanStack Query invalidálás példa (frontend) 35
25.5 AI moderáció példa (backend) 35
Esemény- és jelentkezéskezelés 36
Felhasználókezelés és hitelesítés 39
AI moderáció 40
Tanári időpontfoglalás 41
Összefoglalás 42

Bevezetés

Téma ismertetése

A Blueprint egy teljes stack eseménykezelő alkalmazás, amelyben a felhasználók eseményeket böngészhetnek, jelentkezhetnek, kommentelhetnek, hírfolyamot olvashatnak, valamint tanár-diák konzultációs időpontokat kezelhetnek. A rendszer külön adminisztrációs és tanári munkafolyamatokat is tartalmaz.

A szoftver célja

A Blueprint rendszer elsődleges célcsoportját az oktatási intézmények szereplői alkotják: a diákok, a tanárok és az adminisztratív felhasználók. A platform a diákok számára átlátható esemény- és időpontkezelést biztosít, a tanárok számára strukturált konzultációs szervezést tesz lehetővé, míg az adminisztráció számára egységes felügyeleti felületet nyújt. Ennek köszönhetően a rendszer különösen hasznos olyan intézményekben, ahol fontos a gyors információáramlás, a szerepköralapú működés és a jól dokumentált folyamatkezelés.

Főbb komponensek:

Frontend: Next.js 16 (App Router), React 19, TypeScript, TanStack Query.

Backend: Node.js + Express API, Prisma ORM.

Adatbázis: MySQL.

Objektumtár: MinIO (képfeltöltésekhez).

Fontos modulok:

esemény- és jelentkezéskezelés,

szerepkör alapú jogosultságkezelés,

email megerősítés és jelszó visszaállítás,

tanári elérhetőség + foglalás,

tanterem alapú útvonaltervezés

Fejlesztői útmutató

Frontend technológiák

Next.js 16: App Router, szervezett route-struktúra, jó fejlesztői UX.

React + TypeScript: komponens alapú UI, típusbiztonság.

TanStack Query: cache, invalidálás, mutációk, API állapotkezelés.

Tailwind CSS 4: gyors UI építkezés, token-alapú stílusozás.

Backend technológiák

Express: egyszerű, átlátható REST API rétegzett middleware-rel.

Prisma: típusos DB-hozzáférés, migrációs workflow.

JWT auth: stateless tokenes azonosítás.

Infrastruktúra

MySQL: relációs modell, konzisztens adatszerkezetek.

MinIO: S3-kompatibilis képek tárolása.

Projekt struktúra

Az alkalmazás két fő runtime folyamatra bontható:

frontend runtime (localhost:3000) - UI, route-ok, komponensek.

backend runtime (localhost:8000) - REST API + auth + business logic.

Adatút:

frontend axios hívások -> backend endpointok

backend -> Prisma kliens -> MySQL

backend -> MinIO (feltöltés / törlés)

Auth út:

login után JWT token cookie-ban,

backend middleware validálja,

req.user objektum létrejön,

controllerek szerepkör alapján engedélyeznek.

Backend

Fájl: backend/index.js

Fontos szerepek

CORS szabályozás (PUBLIC_URL, CORS_ORIGINS, localhost fallback)

JSON parse + cookie parse

global rate limiter

route mountolás (/events, /users, /teacher, /admin/*)

Kiemelt kódrészlet: CORS origin normalizálás

```
const normalizeOrigin = (value) => {
  const trimmed = value.trim().replace(/\/+$/, "");
  if (!trimmed) return null;

  const withProtocol =
    trimmed.startsWith("http://") || trimmed.startsWith("https://")
      ? trimmed
      : `http://${trimmed}`;

  try {
    const parsed = new URL(withProtocol);
    return `${parsed.protocol}//${parsed.host}`;
  } catch {
    return null;
  }
};
```

Ez a minta robusztusabb CORS kezelést ad, mert nem omlik össze rosszul formázott .env értékek esetén.

Hitelesítés és jogosultságkezelés

Fájl: backend/middleware/auth.js

Három kulcs middleware:

authenticateToken (általános user)

authenticateAdminToken

authenticateTeacherToken

Folyamat

Token olvasása cookie-ból, fallback Authorization: Bearer headerből.

JWT verify.

DB user lekérdezése (friss role/status/deletedAt ellenőrzés).

req.user építése.

szerepkör szerinti gatekeeping.

Kiemelt kódrészlet: token és adatbázis-állapot összevonása

```
const dbUser = await prisma.user.findUnique({
  where: { id: decoded.userId },
  select: {
    id: true,
    role: true,
    status: true,
    deletedAt: true,
    classroom: true,
    settingJson: true,
  },
});

if (dbUser.deletedAt || dbUser.status !== "active") {
  return res.status(403).json({
    error: "Access denied",
    message: "Account is not active",
  });
}
```

Előnye: a token payload önmagában nem elég, mert azóta változhatott a szerepkör/státusz.

Adatbázis

Fájl: backend/prisma/schema.prisma

Fő entitások

user

event

registration

notification

teacherReservation

teacherAvailability

news

eventNews

eventComment

users tábla / user modell

```
model user {
  id            Int      @id @default(autoincrement())
  name          String
  email         String   @unique
  classroom     String?
  emailVerified Boolean  @default(false)
  role          String   @default("user")
  status        String   @default("active")
  settingJson   Json?    @default("{}")
  deletedAt     DateTime?
  ...
}
```

Ez egyszerre tárolja:

auth-hoz szükséges állapotot,

szerepkört,

tanári tanterem alapbeállítást,

soft-delete információt.

events tábla / event modell és classroom

```
model event {
  id              Int      @id @default(autoincrement())
  name            String
  description     String   @db.Text
  creator         String
  location        String
  classroom       String   @default("Ismeretlen")
  date            DateTime
  maxParticipants Int?
  ...
}
```

Az event.classroom most már core üzleti adat, mert erre épül a helyszínfül navigációs útvonala.

API végpontok és route-struktúra

7.1 Public/User route-ok

Fájl: backend/routes/users.js

POST /users regisztráció

POST /users/login belépés

POST /users/logout

POST /users/email-confirmation/request

POST /users/email-confirmation/confirm

POST /users/password-reset/request

POST /users/password-reset/confirm

GET /users/profile

PUT /users/profile

DELETE /users/profile

GET /users/teachers

7.2 Event route-ok

Fájl: backend/routes/events.js

GET /events

POST /events/:eventId/register

DELETE /events/:eventId/register

komment route-ok

event news route-ok

7.3 Teacher route-ok

Fájl: backend/routes/teacher.js

profil

saját appointmentek

availability CRUD

teacher event létrehozás

7.4 Admin route-ok

backend/index.js alapján:

/admin/events

/admin/users

/admin/appointments

/admin/teacher-availability

/admin/news

/admin/notifications

Eseménykezelés

8.1 Event lista + userhez kötött állapot

Az event listázási endpoint a frontend oldalon megjeleníthető adatokat adja vissza: kapacitás, user regisztrált-e stb.

8.2 Jelentkezési szabályok

registration modell egyedi indexe:

@@unique([userId, eventId])

Ez megakadályozza a duplikált aktív jelentkezéseket. A business logika a státusz alapú reaktivációt is kezelheti (registered, cancelled, attended).

8.3 Kommentek AI moderációval

Fájl: backend/controllers/eventController.js

Kulcs érték: az AI válasz explicit JSON formátumba kényszerített ({"isVerified": boolean, "reason": string}), fallback mechanizmussal.

Kiemelt prompt-részlet:

text: 'You are a strict moderator for event discussion comments. ... Return ONLY valid JSON: {"isVerified": boolean, "reason": string}.'

Előny:

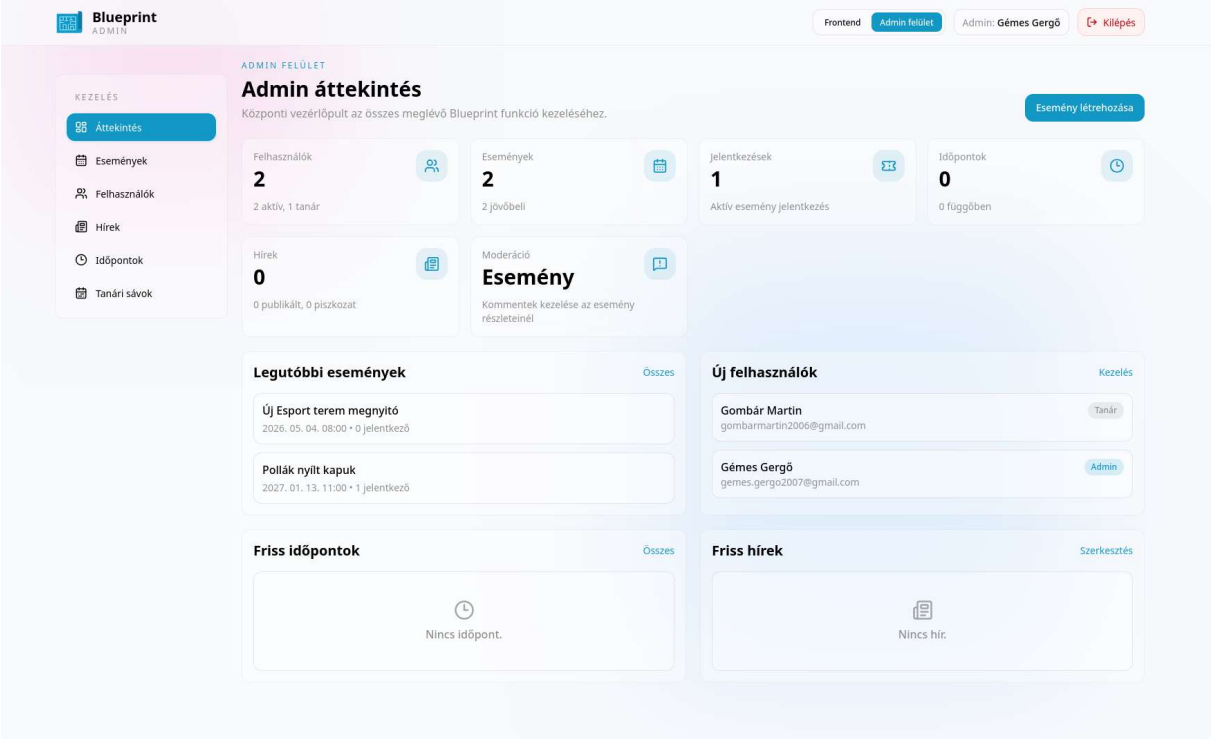
automatizált premoderáció,

transzparens indoklás (reason),

fallback ellenállás API hiba esetén.

Admin felület

Fájl: backend/controllers/adminController.js



Fő modulok:

user adminisztráció,

event CRUD,

appointment adminisztráció,

teacher availability admin nézet.

9.1 Fontos minta: teacher classroom feloldás appointmenthez

```
const getTeacherClassroom = async (teacherId) => {
  const teacher = await prisma.user.findUnique({
    where: { id: Number(teacherId) },
    select: { id: true, role: true, classroom: true },
  });

  if (!teacher || teacher.role !== "teacher") {
    return null;
  }

  return teacher.classroom ?? null;
};
```

Ez biztosítja, hogy időpontfoglalásnál konzisztensen a tanárhoz rendelt tanterem jelenjen meg.

9.2 Event create/update követelmények

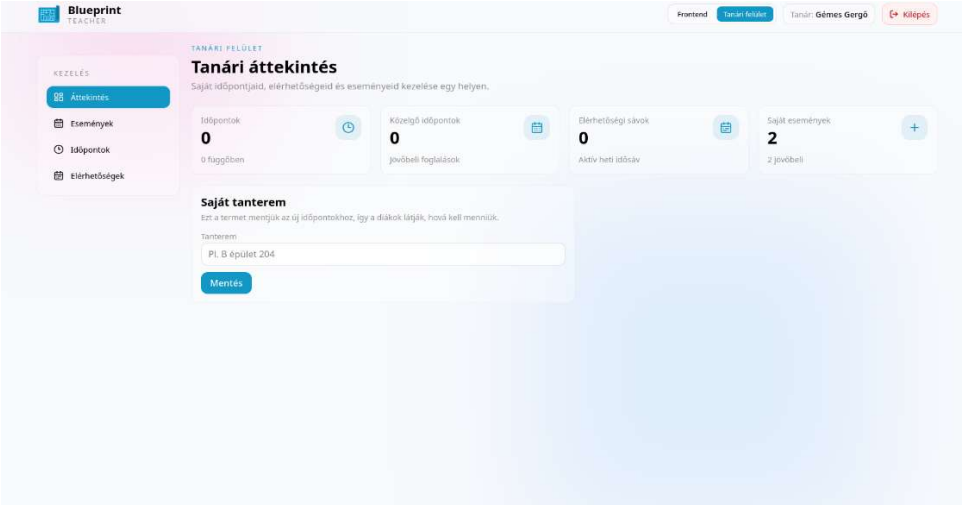
Az aktuális üzleti szabály szerint event létrehozásnál kötelező:

name, description, location, date, creator, classroom.

Ez kulcs, mert a frontend helyszínfüle a classroom alapján számol útvonalat.

Tanári felület

Fájl: backend/controllers/teacherController.js



10.1 Teacher profil

lekérdezés: getTeacherProfile

módosítás: updateTeacherProfile (classroom kötelező bemenet)

10.2 Saját időpontok

Mapelt minta (mapTeacherAppointment) biztosítja, hogy a frontend stabil formátumot kapjon.

10.3 Availability szabályosság

validateAvailabilityPayload szolgáltatással történik az időintervallum validáció.

10.4 Teacher event létrehozás

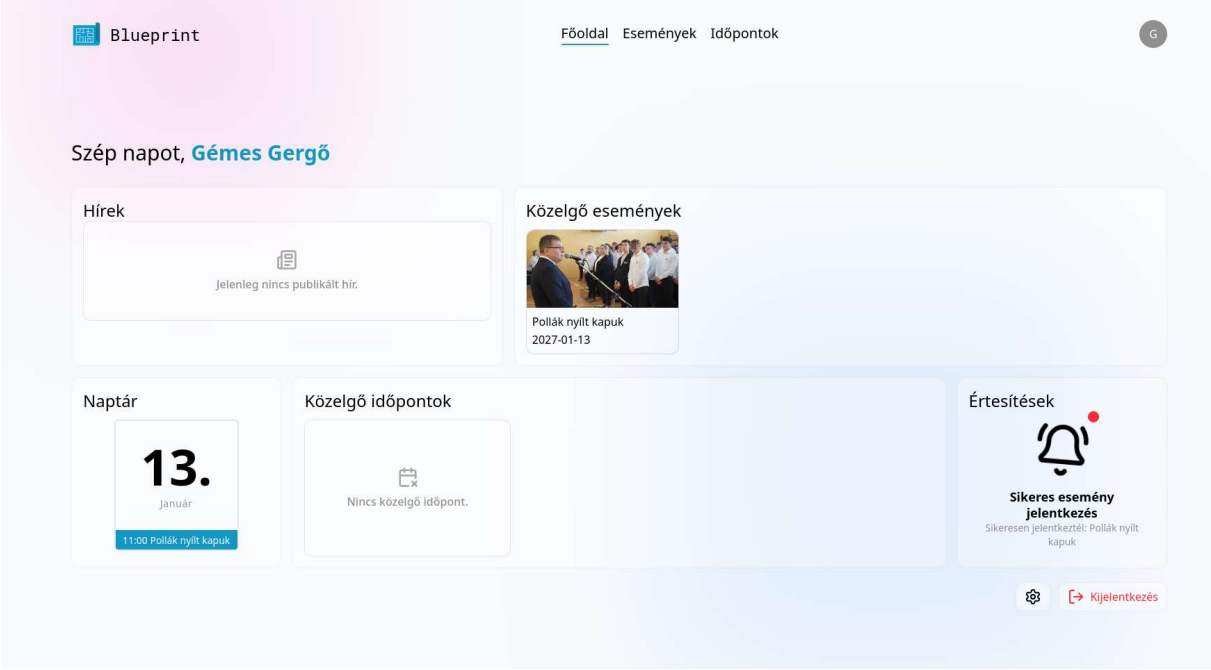
Teacher szerepkör saját eventet tud nyitni, image upload middleware-rel.

Frontend

11.1 App Router szemlélet

A frontend route-ok domain alapján szeparáltak:

(main) publikus/felhasználói app terület



(admin) admin felület

(teacher) tanári felület

(login) auth oldalak

11.2 State és adatkezeles

API kommunikáció: axios

query/mutation/cache: @tanstack/react-query

invalidálás: célzott query key alapján

EventDetailsView komponens

Fájl: frontend/src/app/(main)/app/events/[id]/EventDetailsView.tsx

12.1 Tab-struktúra

export type EventTab = "details" | "news" | "place" | "discussion";

Külön füleken kezeli:

részletek,

hírek,

helyszín (navigáció),

beszélgetés.

12.2 React Query minta

Adatforrások külön query key-ken:

events/detail/{id}

event-news/{id}

event-comments/{id}

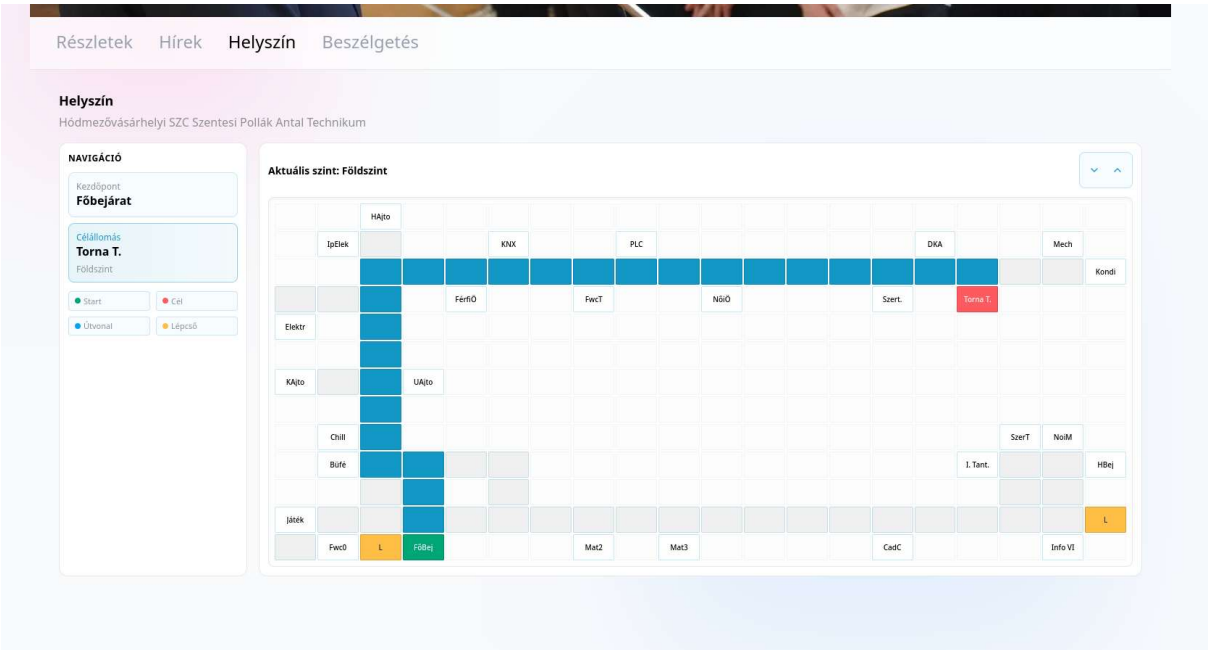
Mutációk után invalidálás történik, pl. komment létrehozásnál:

```
onSuccess: () => {
  setCommentContent("");
  queryClient.invalidateQueries({ queryKey: ["event-comments", eventId] });
}
```

Navigációs modul és útvonaltervezés

Fájl: frontend/src/components/navigation/EventNavigationMap.tsx

Ez a projekt egyik legösszetettebb egyedi modulja.



13.1 Koncepció

az épület raszterre van vetítve (FLOORS mátrix),

minden cella lehet fal, folyosó, terem, lépcső,

fix indulási hely: FőBej,

cél: event.classroom.

13.2 Mátrix reprezentacio

```
type Node = { floor: number; row: number; col: number };
type FloorDef = { rows: number; cols: number; cells: string[][] };
```

Minden szint explicit tömbbel le van írva. Ez nagyobb kódméretet eredményez, de teljes kontrollt ad a fal- és teremtopológia felett.

13.3 Név normalizálás és alias mapping

```
const normalizeName = (value: string) =>
  value
    .trim()
    .toLowerCase()
    .normalize("NFD")
    .replace(/[\u0300-\u036f]/g, "")
    .replace(/\./g, "")
    .replace(/\s+/g, " ");
```

Ez biztosítja, hogy az ékezetek, pontok, whitespace variációk ne tegyék tönkre a lookupot.

13.4 Lépcső kapcsolatok

STAIR_CONNECTIONS explicit grafikussággal írja le a szintek közötti átjárást.

13.5 Blokkolási logika

```
if (value === "X") return true;
if (value === STAIR) return false;
if (value === "") return false;
if (PASSABLE_ROOM_PRESETS.has(value)) return false;
if (value === startName || value === endName) return false;
return true;
```

Ez különbséget tesz fal, folyosó, átjárható terem és fix start/cél pontok között.

13.6 A* heuristika

```
const heuristic = (a: Node, b: Node) => {
  if (STAIR_CONNECTIONS[keyOf(a)]) {
    return Math.abs(a.floor - b.floor);
  }
  return (
    Math.abs(a.row - b.row) +
    Math.abs(a.col - b.col) +
    Math.abs(a.floor - b.floor) * 10
  );
};
```

Látható, hogy emeletkülönbségre erős büntetés van (*10), így a rendszer alapvetően ugyanazon szinten keres utat, és csak szükség esetén vált emeletet.

13.7 UI/UX jellemzők

responsive elrendezés,

emelettel váltás,

térkép jelmagyarázat,

magyar nyelvű,

teljes tanteremnevek oldalsó panelen,

X karakter elrejtése nem bejárható pontokon.

Tanterem-kezelés és jelölési rendszer

Fájl: frontend/src/lib/classrooms.ts

Külön választja:

technikai kódokat (Mat1, VI. Tant.),

emberbarát labelt (Matematika 1, VI. tanterem).

Kiemelt részlet:

```
export const CLASSROOM_LABELS: Record<string, string> = {
  "VI. Tant.": "VI. tanterem",
  Mat1: "Matematika 1",
  CadC: "CAD/CAM labor",
  ...
};
```

Ennek eredménye:

backendben stabil, rövid kódok maradnak,

frontendben olvasható, hosszú nevek jelennek meg.

Biztonság és megbízhatóság

15.1 Rate limiting

global limiter a teljes API-ra,

külön auth limiter a login/regisztráció jellegű endpointokra.

15.2 Email verification gate

Login csak megerősített emaillel engedett.

15.3 Soft delete és status gate

deletedAt + status ellenőrzés auth middleware-ben minden védett hívásnál.

15.4 AI moderation fallback

Ha AI szolgáltatás hiba miatt kiesik, a rendszer nem omlik össze; kontrollált fallback útvonal van.

Hibakezelés

16.1 Backend

konzisztens HTTP status kódok (400, 401, 403, 404, 409, 500)

hibatípus + message mezők a JSON válaszokban

16.2 Frontend

loading állapot (Spinner)

hibaállapot külön üzenettel

mutációs pending állapotok disable-elik a gombokat

Teljesítményi szempontok

17.1 Query invalidálás

Célzott query key invalidálások minimalizálják a felesleges újrakereséseket.

17.2 Navigáció algoritmus

mátrix méret fix és viszonylag kicsi,

kliens oldali számítás valós időben kezelhető,

preindexelt ROOM_INDEX gyors lookupot ad.

17.3 Prisma

szelektív select használat auth middleware-ben,

indexelt mezők a schema-ban (pl. notification, availability, eventComment).

A rendszer telepítése és futtatása

Frontend és backend indítása

backend és frontend npm install

backend .env kitöltése

npx prisma generate

npx prisma migrate deploy

backend: npm run dev

frontend: npm run dev

Adatbázis migráció és Prisma frissítés

Mindig szükséges:

npx Prisma generate

Ha ez kimarad, tipikus runtime hiba lehet: Unknown field ... for select statement.

Tervezési döntések és indoklás

19.1 Miért mátrix alapú navigáció?

Előny:

determinisztikus,

könnyen vizualizálható,

Hátránya:

manuális karbantartási igény (térképváltozásnál kódmódosítás).

19.2 Miért kötelező classroom eventnél?

Előny:

minden eseményhez van útvonaltervezhető célpont,

nincs bizonytalan helyszínfüle.

19.3 Miért role + status + deletedAt gate?

Előny:

jogosultság dinamikusan visszavonható,

kompromittált tokenek hatása csökkenthető.

Kódrészlet-gyűjtemény

21.1 Route mountolás (backend)

```
app.use("/events", eventRoutes);
app.use("/users", userRoutes);
app.use("/teacher", teacherRoutes);
app.use("/admin/events", adminEventsRoutes);
```

21.2 Auth user gate

```
if (dbUser.deletedAt || dbUser.status !== "active") {
  return res.status(403).json({
    error: "Access denied",
    message: "Account is not active",
  });
}
```

21.3 Event classroom kötelezettség

```
if (!name || !description || !location || !date || !normalizedCreator ||
!normalizedClassroom) {
  return res.status(400).json({
    error: "Missing required fields",
    message: "Name, description, location, date, creator, and classroom are required",
  });
}
```

21.4 A* útvonalkeresés mag

```
const tentative = (gScore[currentKey] ?? Infinity) + 1;
if (tentative < (gScore[nextKey] ?? Infinity)) {
  came[nextKey] = current;
  gScore[nextKey] = tentative;
  fScore[nextKey] = tentative + heuristic(next, end);
}
```

Fogalomtár

classroom: tanterem kód, ami eseményhez/tanárhoz rendelhető.

eventMap preset: korábbi statikus térkép preset koncepció (kivezetve).

availability: heti ismétlődő tanári idősáv.

teacherReservation: konkrét foglalt időpont két fel között.

eventNews: eseményhez kötött hír, draft/published állapottal.

eventComment.isVerified: AI moderációs állapot.

Tesztelés

Tesztelési stratégia

A projektben két szintű tesztelés történik: backend oldali automatizált egységtesztek és frontend oldali Playwright e2e smoke tesztek.

A backend tesztek a node:test futtatót használják, mockolt HTTP válaszobjektumokkal és célzott Prisma/Bcrypt/JWT stubokkal.

A frontend tesztek útvonalvédelemre, bejelentkezési folyamatra és szerepkörfüggő hozzáférésre koncentrálnak.

Backend automatizált tesztek

Tesztek helye: backend/tests/*.test.js

Futtatási parancs: cd backend && npm test

A futtatás eredménye a dokumentáció készítésekor: 12/12 sikeres teszt, 0 hiba.

Lefedett tipikus backend esetek

1) authenticateToken: token nélküli kérés esetén 401 és "No token provided" válasz.

2) authenticateToken: inaktív felhasználó esetén 403 és "Account is not active".

3) authenticateAdminToken: nem admin szerepkör esetén 403 és "Admin privileges required".

4) authenticateAdminToken: admin felhasználó esetén next() hívás és req.user admin adatok beállítása.

5) userLogin: törölt (deletedAt) fiók blokkolása jelszó-ellenőrzés előtt.

6) userLogin: inaktív fiók blokkolása jelszó-ellenőrzés előtt.

7) userLogin: aktív felhasználó sikeres belépésekor token cookie létrehozása.

8) confirmEmail: érvényes email-token esetén felhasználó státusz frissítése.

9) resetPassword: érvénytelen token esetén 400-as hiba.

10) getEventRegistrations: nem numerikus eventId esetén 400-as hiba.

11) getEventRegistrations: nem admin hozzáférés tiltása.

12) getEventRegistrations: admin esetén regisztrációs lista sikeres visszaadása.

Frontend e2e smoke tesztek (Playwright)

Tesztek helye: frontend/e2e/*.spec.ts és frontend/e2e/setup/auth.setup.ts

Futtatási parancsok:

- cd frontend && npm run test:e2e

- cd frontend && npm run test:e2e:auth

Főbb ellenőrzött e2e esetek

1) Login oldal kötelező mezőinek megléte (email, jelszó, bejelentkezés gomb).

2) "Elfelejtett jelszó" útvonal elérhetősége a bejelentkezési oldalról.

3) Védett /events útvonal átirányítása login oldalra anonim felhasználónál.

4) Védett /appointments útvonal átirányítása login oldalra anonim felhasználónál.

5) Védett /teacher/events útvonal átirányítása login oldalra anonim felhasználónál.

6) Hitelesített felhasználó számára /events és /appointments oldalak elérhetősége.

7) Hitelesített tanár számára /teacher/events oldal és "Új esemény" gomb ellenőrzése.

Tesztelői hozzáférés és előfeltételek

A Playwright hitelesített tesztek környezeti változókból olvassák a tesztfiókok adatait:

PLAYWRIGHT_USER_EMAIL, PLAYWRIGHT_USER_PASSWORD, PLAYWRIGHT_TEACHER_EMAIL, PLAYWRIGHT_TEACHER_PASSWORD.

A setup teszt automatikusan létrehozza a storage state fájlokat (frontend/e2e/.auth/user.json és teacher.json), így a szerepkörös e2e futások reprodukálhatók.

Hibakezelési fókusz

A tesztcsomag külön figyel a hozzáférés-vezérlési és állapotfüggő hibákra (token hiány, inaktív/törölt fiók, nem admin hozzáférés, hibás bemeneti azonosító), így a biztonsági és jogosultsági regressziók gyorsan észlelhetők.

Fontosabb fájlok röviden

24.1 Backend- kulcsfájlok

backend/index.js - API belépési pont; CORS, middleware-lánc, route mountolás.

backend/prisma/schema.prisma - teljes adatmodell (user/event/registration/news/comment/foglalás).

backend/middleware/auth.js - JWT ellenőrzés + role/status gatekeeping (user, teacher, admin).

backend/controllers/userController.js - regisztráció, login, profil, email confirm, password reset.

backend/controllers/eventController.js - event listázás, jelentkezés, kommentek, event hírek, AI moderáció.

backend/controllers/adminController.js - admin CRUD-k (userek, események, foglalások, statuszok).

backend/controllers/teacherController.js - tanári profil, availability, időpontok, teacher eventek.

backend/services/emailService.js - email kuldes (auth flowkhoz).

backend/services/teacherAvailabilityService.js - availability validáció/slot logika.

backend/middleware/upload.js - képek feltöltése + MinIO összekötés.

24.2 Frontend- kulcsfájlok

frontend/src/app/layout.tsx - root layout + global provider bekötés.

frontend/src/contexts/AuthContext.tsx - auth állapot, user session, role alapú UI gate.

frontend/src/types.d.ts - API DTO-k és kliensoldali típus rendszer.

frontend/src/app/(main)/app/events/[id]/EventDetailsView.tsx - event részletoldal (details/news/place/discussion tabok).

frontend/src/components/navigation/EventNavigationMap.tsx - A* + floors mátrix.

frontend/src/lib/classrooms.ts - tanterem opciók, label mapping, fix navigációs start.

frontend/src/app/(admin)/admin/events/page.tsx - admin event kezelés (kötelező classroom, feltöltés).

frontend/src/app/(teacher)/teacher/events/page.tsx - teacher event létrehozás (kötelező classroom).

frontend/src/app/globals.css - global theme tokenek és alap stilusok.

24.3 Route fájlok (fontosabb)

backend/routes/users.js - auth + profil endpointok.

backend/routes/events.js - event, regisztráció, komment, event news endpointok.

backend/routes/teacher.js - teacher oldali endpointok.

backend/routes/admin/adminEvents.js - admin event route-ok.

backend/routes/admin/adminUsers.js - admin user route-ok.

24.4 Dokumentáció

README.md - projekt gyorstalpaló.

backend/README.md - backend setup és API.

frontend/README.md - frontend setup és route áttekintés.

docs/PROJEKT_DOKUMENTACIO_HU.md - részletes szakmai dokumentáció (ez a Fájl).

Mini működési példák

25.1 Auth gate példa (backend)

Mi történik? A token ellenőrzése után a rendszer DB-bol újraolvassa a felhasználó aktuális státuszát/szerepkörét.

```
const dbUser = await prisma.user.findUnique({
  where: { id: decoded.userId },
  select: { role: true, status: true, deletedAt: true },
});

if (dbUser.deletedAt || dbUser.status !== "active") {
  return res.status(403).json({ error: "Access denied" });
}
```

Miért fontos? Ha valakit közben tiltották/inaktiválták, régi tokennel sem léphet be.

25.2 Event classroom követelmény példa

Mi történik? Event létrehozásnál a classroom kötelező, mert erre épül a helyszín navigáció.

```
if (!name || !description || !location || !date || !normalizedClassroom) {
  return res.status(400).json({
    message: "Name, description, location, date, creator, and classroom are required",
  });
}
```

Miért fontos? Nincs "üres" helyszínfüle: minden eventhez tervezhető útvonal.

25.3 A* útvonaltervezés példa (frontend)

Mi történik? A rendszer raszteren keres utat, figyelembe veszi a falakat, lépcsőket, szinteket.

```
const tentative = (gScore[currentKey] ?? Infinity) + 1;
if (tentative < (gScore[nextKey] ?? Infinity)) {
  came[nextKey] = current;
  gScore[nextKey] = tentative;
  fScore[nextKey] = tentative + heuristic(next, end);
}
```

Miért fontos? Determinisztikus, gyors, és vizuálisan jól követhető.

25.4 TanStack Query invalidálás példa (frontend)

Mi történik? Komment létrehozás után csak a releváns query frissül.

```
onSuccess: () => {
  setCommentContent("");
  queryClient.invalidateQueries({ queryKey: ["event-comments", eventId] });
}
```

Miért fontos? Friss UI teljes oldal reload nélkül.

25.5 AI moderáció példa (backend)

Mi történik? A kommenthez AI verifikáció fut, standard JSON választ vár.

```
systemInstruction: {
  parts: [{
    text: 'Return ONLY valid JSON: {"isVerified": boolean, "reason": string}.',
  }],
}
```

Miért fontos? Moderációs döntés strukturáltan kerül vissza, fallback kezeléssel.

Esemény- és jelentkezéskezelés

A jelentkezési rendszer biztosítja, hogy egy eseményre csak egyszer lehessen feliratkozni, a férőhelyek száma pedig ne léphető túl.

A szerepkör ellenőrzése után a backend ellenőrzi, hogy a felhasználó már regisztrált-e, majd létrehozza a jelentkezési rekordot vagy jelzi, hogy a férőhely megtelt.

Lemondáskor a rendszer frissíti a státuszt és felszabadítja a férőhelyet.

```
async function registerForEvent(req, res) {
  const { eventId } = req.params;
  const userId = req.user.id;

  // keresd meg az eseményt
  const event = await prisma.event.findUnique({ where: { id: Number(eventId) } });
  if (!event) return res.status(404).json({ error: 'Event not found' });

  // ellenőrizd a duplikált regisztrációt
  const existing = await prisma.registration.findUnique({
    where: { userId_eventId: { userId, eventId: Number(eventId) } },
  });
  if (existing) return res.status(400).json({ error: 'Already registered' });

  // ellenőrizd a férőhelyeket
  if (event.maxParticipants && event.maxParticipants <= 0) {
    return res.status(400).json({ error: 'Event is full' });
  }

  // hozd létre a regisztrációt
  await prisma.registration.create({
    data: {
      userId,
      eventId: Number(eventId),
      status: 'registered',
      registrationDate: new Date(),
    },
  });

  // küldj értesítő emailt
  await sendRegistrationEmail(userId, event);

  res.json({ message: 'Registered successfully' });
}
```

```
async function unregisterFromEvent(req, res) {
  const { eventId } = req.params;
  const userId = req.user.id;

  const registration = await prisma.registration.findUnique({
    where: { userId_eventId: { userId, eventId: Number(eventId) } },
  });
  if (!registration) return res.status(404).json({ error: 'Not registered' });

  await prisma.registration.update({
    where: { id: registration.id },
    data: { status: 'cancelled' },
  });

  res.json({ message: 'Registration cancelled' });
}
```

Felhasználókezelés és hitelesítés

A felhasználókezelés és hitelesítés a regisztráció, bejelentkezés, profilmódosítás és email-megerősítés folyamatait fedi le.

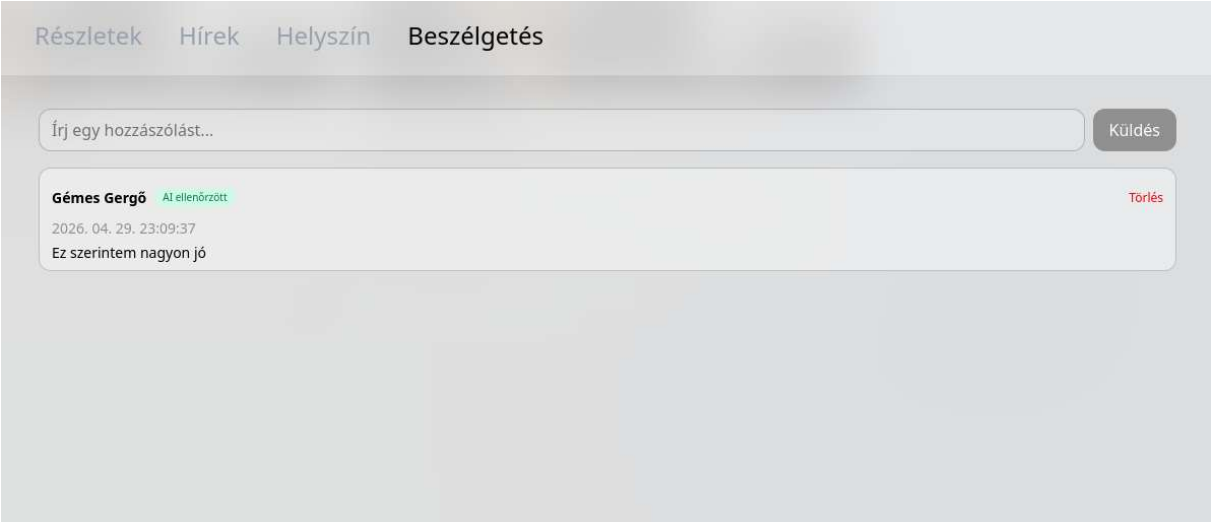
Az alábbi kódrészlet bemutatja, hogyan történik a regisztráció és a bejelentkezés a backendben. A jelszavak hash-elése Bcrypttel történik, majd JWT tokent kap a felhasználó.

```
async function createUser(req, res) {
  const { name, email, password, classroom } = req.body;
  const hashed = await bcrypt.hash(password, 10);
  const user = await prisma.user.create({
    data: { name, email, password: hashed, classroom, role: 'user', status: 'active' },
  });
  // send verification email
  await sendEmailConfirmation(user);
  res.status(201).json({ message: 'User created' });
}

async function loginUser(req, res) {
  const { email, password } = req.body;
  const user = await prisma.user.findUnique({ where: { email } });
  if (!user) return res.status(401).json({ error: 'Invalid credentials' });
  const valid = await bcrypt.compare(password, user.password);
  if (!valid) return res.status(401).json({ error: 'Invalid credentials' });
  const token = jwt.sign({ userId: user.id }, process.env.JWT_SECRET, { expiresIn:
'1d' });
  res.cookie('token', token, { httpOnly: true, secure: true });
  res.json({ message: 'Logged in' });
}
```

AI moderáció

Az eseménykommentek moderációját egy külső AI szolgáltatás végzi. A moderator szigorú promptot kap, és csak érvényes JSON-t kell visszaadnia.



Ha a válasz nem parse-olható JSON, vagy nincs API-kulcs konfigurálva, a rendszer fallback módba kapcsol.

```
async function verifyCommentWithAI(comment) {
  const systemInstruction = {
    parts: [{
      text: 'You are a strict moderator for event discussion comments. Check if content
is appropriate and return ONLY valid JSON: {"isVerified": boolean, "reason": string}.',
    }],
  };
  const userMessage = { role: 'user', parts: [{ text: comment }] };
  try {
    const result = await model.generateContent([systemInstruction, userMessage]);
    const parsed = JSON.parse(result.choices[0].message.content);
    return parsed;
  } catch (error) {
    return { isVerified: false, reason: 'Fallback: AI service failed' };
  }
}
```

Tanári időpontfoglalás

A tanári időpontfoglalás lehetővé teszi, hogy a diákok egy tanár előre meghatározott idősávjaiban foglaljanak időpontot.

A backend ellenőrzi, hogy a tanár azonosítója érvényes, a kért intervallum belefér az elérhetőségbe, és hogy nincs-e ütközés más foglalásokkal.

```
async function createAppointment(req, res) {
  const { teacherId, title, startDate, endDate } = req.body;
  const teacher = await prisma.user.findUnique({ where: { id: Number(teacherId) } });
  if (!teacher || teacher.role !== 'teacher') {
    return res.status(404).json({ error: 'Teacher not found' });
  }
  // check availability overlap
  const overlapping = await prisma.teacherReservation.findFirst({
    where: {
      teacherId: teacher.id,
      OR: [
        { startDate: { lte: new Date(endDate) }, endDate: { gte: new Date(startDate) } },
      ],
    },
  });
  if (overlapping) return res.status(409).json({ error: 'Time slot unavailable' });
  const reservation = await prisma.teacherReservation.create({
    data: { teacherId: teacher.id, title, startDate: new Date(startDate), endDate: new
Date(endDate), userId: req.user.id },
  });
  res.json(reservation);
}
```

Összefoglalás

A Blueprint jelenlegi állapota egy funkcionálisan erős, domain-szinten jól szeparált rendszer:

korszerű frontend + strukturált backend,

role/status alapú auth,

classroom-kozpontu esemény és navigáció modell,

admin/teacher workflow különválasztás,

AI-t támogató moderációs képességek.

Ez a dokumentum a fejlesztői onboardingra, belső tudástranszferre, valamint formálisabb projektátadásra is alkalmas alap.